rbus

локальная высокопроизводительная шина сообщений для автомобильных и робототехнических систем

Проблема

Нужно быстро передавать данные между процессами на одной машине

- Универсальные robotics-фреймворки часто рассчитаны на широкий класс систем
- В них есть сетевые сценарии, разные режимы доставки и много runtime-настроек
- В наших задачах все процессы работают на одном Linux-устройстве
- Поэтому главный вопрос проще: как сделать быстрый локальный IPC без лишних копий

rbus не заменяет универсальный middleware целиком

Это локальный IPC/SHM-транспорт для среза, где участники находятся на одном устройстве

Цели проекта

Сделать локальный fast path для publish/subscribe

- **Производительность** — shared memory, loaned payload без копирования, non-blocking hot path без syscall и heap allocation
- **Простая модель и локальный scope** — минимальный Rust API, одна машина, один daemon registry, без попытки заменить универсальный middleware
- **Надёжность** — cleanup при падении процессов, closed stream через shared memory, restart daemon-а не ломает уже attached data plane
- **Диагностика** — rbus-cli, doctor, stats, warnings, inspect, benchmark smoke

Требования

Что должен уметь текущий этап

Функциональные

- несколько publisher-ов и subscriber-ов на topic
- fan-out доставки одному и нескольким consumer-ам
- blocking и non-blocking receive
- контрольный daemon и CLI-диагностика
- cleanup после смерти publisher/subscriber

Нефункциональные

- Linux-only, локальная машина
- payload не проходит через daemon
- 0 syscalls/message для non-blocking data path
- 0 allocations/message в zero-copy hot path
- ABI/version contracts и воспроизводимые бенчмарки

Границы проекта

Что входит в текущий этап

Локальный publish/subscribe, shared-memory ring, FD passing, daemon registry, warning model, CLI, benchmark harness

Что не входит

Сетевой транспорт, сложные политики доставки, async/callback API, serde-схемы, C/Python bindings, автоматический запуск daemon-а

Что сравниваем

Unix socket baseline против rbus SHM data path

- **Одинаковый scope** — локальный IPC на одной Linux-машине
- **Unix socket** — payload проходит через kernel socket path и socket buffer
- **rbus** — payload лежит в shared-memory ring, loaned path не копирует payload
- **Что проверяем** — что даёт переход от socket baseline к SHM data path

Архитектура: общий вид

Daemon управляет ресурсами, payload идёт напрямую

Control plane

Publisher process ↔ rbusd ↔ Subscriber process

- REGISTER / ATTACH / INSPECT через Unix SOCK_SEQPACKET
- FD передаются через SCM_RIGHTS
- daemon владеет registry и lifecycle

Data plane

Publisher stream memfd/mmap MAP_SHARED → Subscriber cached pointers

- daemon не копирует payload
- daemon не маршрутизирует payload
- после attach hot path работает через shared memory

Архитектура: control plane

rbusd нужен для setup, registry и cleanup

- **REGISTER_PUBLISHER** создаёт memfd stream segment и выдаёт publisher_id, stream_id, FD
- **REGISTER_SUBSCRIBER** выдаёт subscriber_id и metadata для attach
- **ATTACH_STREAMS** передаёт stream FD через SCM_RIGHTS
- **NOTIFY_STREAM_ADDED** добавляет новые publisher stream-ы без refresh в steady state
- persistent control session связывает registry entries с процессом

Архитектура: data plane

Один publisher stream — это fixed-size SPMC ring

Memory layout

- SegmentHeader
- StreamCounters
- SlotHeader[depth]
- PayloadSlot[depth]

Slot protocol

- writer получает slot и пишет payload
- commit публикуется release-store в state_seq
- reader pin-ит slot acquire/release CAS-ом
- MessageRef освобождает reader bit при Drop
- blocking recv ждёт topic-level futex segment

Публичный API

Минимальные типы, без лишней runtime-модели

- **Типы** — Bus, TopicConfig, Publisher, Subscriber, LoanedMessage, MessageRef, Stats, Warning
- **Сценарий** — connect → publisher/subscriber → loan → write payload → publish → receive
- **Контракты** — public API 0.1.0, stream ABI 5, topic wait ABI 1

Гарантии hot path

Главные обещания data path

0

heap allocations: loan + publish

0

syscalls/message: non-blocking publish/try_recv

0

payload copies в loaned path

Как проверяется

- custom allocator tests
- marker-based strace smoke
- Loom-модели atomic protocol
- layout tests на cache-line counters

Технологии

Минимальный Linux/Rust стек

- **Rust 2024** — workspace package 0.1.0, warnings=deny, dead_code=deny
- **Linux IPC** — memfd_create, ftruncate, mmap MAP_SHARED, Unix SOCK_SEQPACKET, SCM_RIGHTS
- **Синхронизация** — futex для blocking recv, acquire/release atomics для slot protocol
- **Benchmarking** — perf stat, strace, CSV/JSON reports, regression gates

Методика измерений

Окружение и режимы нагрузки

- Ubuntu 26.04 LTS, Linux 7.0.0-27-generic
- Intel Core i7-11700F: 8C/16T
- governor: powersave, intel_pstate active
- cargo build –profile bench
- rbusd и rbus-bench из target/release

Режимы

- saturated / busy-poll: 4 publisher-а, 8 subscriber-ов
- payload: 64 B, 4 KiB, 64 KiB, 1 MiB
- paced / 30 Hz: controlled-load latency
- low-CPU recv: 5 секунд, 150 сообщений, 1P1S

Результаты: throughput

Saturated busy-poll, 4P8S, payload 64 B



5.7x

loaned-fill vs unix socket

0

syscalls/message в rbus busy-poll data path

0

allocations/message в rbus busy-poll hot path

Результаты: большие payload

Saturated busy-poll, effective delivery GiB/s

Payload	loaned	publish	unix	x
4 KiB	53.8	59.1	8.2	6.5x
64 KiB	120.2	92.1	21.8	5.5x
1 MiB	146.1	154.1	15.6	9.4x

metadata-only

1 MiB: 661.2 GiB/s logical/effective

Это commit/control overhead без записи payload в measured window

Результаты: задержка под нагрузкой

Paced 30 Hz, 1P1S, busy-poll, p99 transport latency

Payload	loaned	publish	unix	x
64 B	1.2 us	2.4 us	126.7 us	105x
4 KiB	8.0 us	9.2 us	151.7 us	19x
64 KiB	23.5 us	23.8 us	276.1 us	12x
1 MiB	23.2 us	110.2 us	1615.5 us	70x

- Этот режим показывает нижнюю границу latency
- Он использует busy-poll и почти целое ядро на worker path
- Metadata-only не используется как основной latency-result

Результаты: низкий CPU в recv

Paced 30 Hz, 1P1S, wait-mode recv

64 B

- rbus loaned-fill: p99 81 us, 0.004 cores
- unix-socket: p99 103 us, 0.001 cores
- вывод: rbus быстрее по latency, Unix дешевле по CPU

1 MiB

- rbus loaned-fill: p99 231 us, 0.014 cores
- unix-socket: p99 1369 us, 0.019 cores
- вывод: rbus ниже по latency при сопоставимом CPU

Результаты: корректность

Benchmark-и не должны скрывать ошибки доставки

0

warning_count в выбранных строках

0

skipped_messages

0

publish deadline misses

Тесты

- cross-process data plane tests
- examples suite
- subscriber crash с удержанным MessageRef
- publisher crash
- rbusd restart

Сильные стороны и ограничения

Что получилось и где границы

Сильные стороны

- payload не проходит через daemon
- zero-copy loaned path
- высокий throughput на локальной машине
- microsecond-level paced latency в busy-poll
- низкий CPU recv path через futex

Ограничения

- Linux-specific IPC primitives
- API пока Rust-only
- нет сетевого транспорта
- один stream ограничен 63 subscriber reader bits
- low-CPU small payload не всегда дешевле Unix socket по CPU

Выводы

rbus закрывает локальный IPC-срез проекта

- **Цель достигнута** — реализован локальный message bus, где setup делает daemon, а payload идёт через shared memory
- **Ниша понятна** — fast local transport для процессов на одной машине, не замена универсального middleware
- **Результат измерен** — есть throughput, latency, perf/strace данные и regression tests

Следующие шаги

- adapter для внешнего robotics stack
- стабилизация API
- C++ binding для robotics-кода
- реальные sensor payloads
- долгие soak-тесты